

Algoritmos de ordenamiento

Insertion Sort (Ordenamiento por inserción)

- Ordena los elementos de manera ascendente de menor a mayor.
- Toma uno por uno los elementos de un arreglo y lo recorre hacia su posición con respecto a los anteriormente ordenados.
 - > Empieza con el segundo elemento y lo ordena con respecto al primero.
 - > Sigue con el tercero y lo coloca en su posición ordenada con respecto a los dos anteriores, así sucesivamente hasta recorrer todas las posiciones del arreglo.

❖ Algoritmo

```
public class InsertionSort{  
    public static void insertionSort(int array[]) {  
        int n = array.length;  
        for (int j = 1; j < n; j++) {  
            int key = array[j];  
            int i = j-1;  
            while ( (i > -1) && ( array [i] > key ) ) {  
                array [i+1] = array [i];  
                i--;  
            }  
            array[i+1] = key;  
        }  
    }  
}
```

//Llamado del método insertionSort(arr1);

❖ Aplicación

Considerando la siguiente lista de 10 elementos:

20 - 4 - 8 - 11 - 23 - 3 - 5 - 2 - 7 - 14

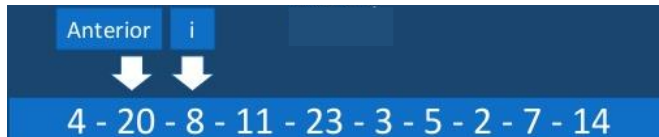
- 1) La comparación se hace desde el segundo elemento de la lista (i), a continuación, se verifica que i sea menor a su antecesor, de ser verdad esta afirmación, i se intercambiará con su antecesor; de lo contrario, si la afirmación es falsa, el segundo elemento no se moverá si no que tomará la posición de su sucesor, sabiendo que el segundo elemento ya está ordenado.



Quedando así:

4 - 20 - 8 - 11 - 23 - 3 - 5 - 2 - 7 - 14

- 2) i tomará la posición de 8, y este se comparará con 20, $8 < 20$ es verdadero, se intercambian entre sí.



Quedando así:

4 - 8 - 20 - 11 - 23 - 3 - 5 - 2 - 7 - 14

Se compara $8 < 4$ lo que es falso, 8 no se moverá porque ya se encuentra ordenado.

- 3) i tomará la posición de 11, y se compara $11 < 20$ lo que es verdadero, se incrementará entre sí.



Quedando así:

4 - 8 - 11 - 20 - 23 - 3 - 5 - 2 - 7 - 14

Se comparará $11 < 8$ y $11 < 4$ lo que es falso, 11 ya no se moverá porque se encuentra ordenado.

- 4) i tomará la posición de 23 y se comparará $23 < 20$ lo cual es falso, no se moverá porque ya se encuentra en orden.
- 5) i tomará la posición de 3, y se comparará $3 < 23$ y $3 < 20$ hasta llegar a $3 < 4$, todas estas comparaciones serán verdaderas y se intercambiarán entre sí, quedando el 3 en la primera posición.



Quedando así:

3 - 4 - 8 - 11 - 20 - 23 - 5 - 2 - 7 - 14

- 6) i tomará la posición de 5, y se comparará $5 < 23$ y $5 < 20$ hasta llegar a $5 < 8$, todas serán verdaderas.



Cuando llegue a $5 < 4$ esto será falso, por lo que 5 quedará antes de 8 y después de 4, quedando así:



- 7) i tomará la posición de 2, y se comparará $2 < 23$ y $2 < 20$ hasta llegar a $2 < 3$, todas las afirmaciones serán verdaderas, y 2 quedará en la primera posición.



Quedando así:



- 8) i tomará la posición de 7, el cual se comparará $7 < 23$ y $7 < 20$ hasta llegar a $7 < 8$, todas estas afirmaciones serán verdaderas. $7 < 5$ es falso, por lo que 7 se quedará antes de 8 y después de 5.



Quedando así:



- 9) i tomará la posición de 14, el cual se comparará $14 < 23$ y $14 < 20$, estas afirmaciones serán verdaderas. Ahora $14 < 11$ es falso, por lo que 14 quedará antes de 20 y después de 11.



Quedando por fin *ordenado*:

2 - 3 - 4 - 5 - 7 - 8 - 11 - 14 - 20 - 23

Shell sort

- Ordena una estructura de una manera similar a la del Bubble Sort.
- No ordena elementos adyacentes, sino que utiliza una segmentación entre los datos.
- La segmentación puede ser de cualquier tamaño de acuerdo a una secuencia de valores que empiezan con un valor grande (pero menor al tamaño total de la estructura) y van disminuyendo hasta llegar al '1'.
- Su complejidad es de $O(n^{1.25})$ en el mejor caso y de $O(n^{1.25})$ en el caso promedio.
- Se ha comprobado que el tiempo de ejecución promedio es de $O(n^{2/3})$ para la mayoría de las secuencias de salto.
- Compara cada elemento con el que está a un cierto número de posiciones llamado salto, en lugar de compararlo con el que está justo a su lado.
 - > El salto es constante, y su valor inicial es $N/2$ (siendo N el número de elementos, y siendo división entera).
 - > Se van dando pasadas con el mismo salto hasta que en una pasada no se intercambie ningún elemento de sitio.
 - > Entonces el salto se reduce a la mitad, y se vuelven a dar pasadas hasta que no se intercambie ningún elemento, y así sucesivamente hasta que el salto vale 1.

❖ Algoritmo

- * Primera opción: recibe un arreglo a ordenar $a[]$ y el tamaño del arreglo n .

```
void shellsort ( int a[], int n)
{
    int x,i,j,inc,s;

    for(s=1; s < t; s++) /* recorre el arreglo de incrementos */
    {
        inc = h[s];
        for(i=inc+1; i < n; i++)
        {
            x = a[i];
            j = i-inc;
            while( j > 0 && a[j] > x)
            {
                a[j+inc] = a[j];
                j = j-inc;
            }
            a[j+inc] = x;
        }
    }
}
```

```
}  
}
```

* Segunda opción: ordenar un array A de enteros es el siguiente:

```
public static void shell(int A[]) {
```

```
    int salto, aux, i;  
    boolean cambios;
```

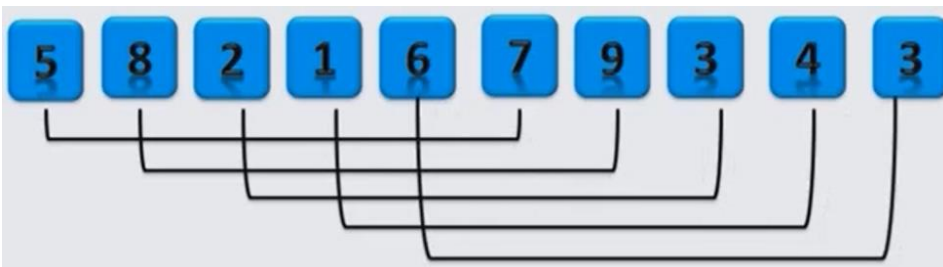
```
    for (salto = A.length / 2; salto != 0; salto /= 2) {  
        cambios = true;  
        while (cambios) { // Mientras se intercambie algún elemento  
            cambios = false;  
            for (i = salto; i < A.length; i++) // se da una pasada  
            {  
                if (A[i - salto] > A[i]) { // y si están desordenados  
                    aux = A[i]; // se reordenan  
                    A[i] = A[i - salto];  
                    A[i - salto] = aux;  
                    cambios = true; // y se marca como cambio.  
                }  
            }  
        }  
    }  
}
```

❖ Aplicación

Considerando el siguiente arreglo de 10 elementos:



- 1) **Primera pasada:** Se dividen los elementos del arreglo en 5 grupos, teniendo en cuenta los elementos que se encuentran a 5 posiciones de distancia entre sí y se ordenan por separado.



Comparaciones:

- $5 < 7 = \text{Verdadero}$
- $8 < 9 = \text{Verdadero}$
- $2 < 3 = \text{Verdadero}$
- $1 < 4 = \text{Verdadero}$
- $6 < 3 = \text{Falso}$

Por lo que 6 se intercambiará de posición con el 3:



2) Se repite el ordenamiento con comparaciones de la misma distancia.

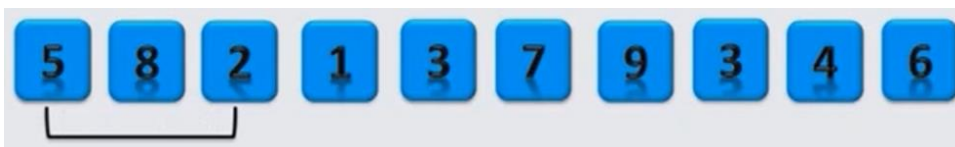
Comparaciones:

- $5 < 7 = \text{Verdadero}$
- $8 < 9 = \text{Verdadero}$
- $2 < 3 = \text{Verdadero}$
- $1 < 4 = \text{Verdadero}$
- $3 < 6 = \text{Verdadero}$

Por lo que no hay intercambios:



3) Se realizarán comparaciones de distancia igual a dos, debido a que no existieron cambios:



- $5 < 2 = \text{Falso}$, se hace un intercambio:



- $5 < 3 = \text{Falso}$, se hace un intercambio:



- $5 < 9 = \text{Verdadero}$, se queda igual:



- $9 < 4$ = Falso, se hace un intercambio:



- $8 < 1$ = Falso, se hace un intercambio:



- $8 < 7$ = Falso, se hace un intercambio:



- $8 < 3$ = Falso, se hace un intercambio:



- $8 < 6$ = Falso, se hace un intercambio:



4) Se repite el ordenamiento con comparaciones de la misma distancia:



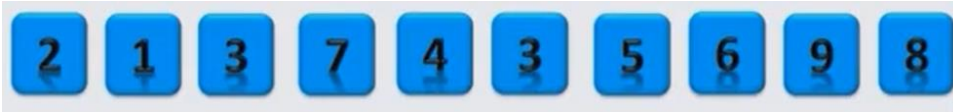
- $2 < 3$ = Verdadero, no se realizan intercambios:



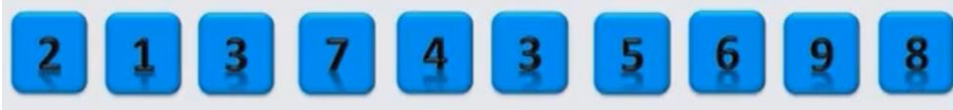
- $3 < 5$ = Verdadero, no se realizan intercambios:



- $5 < 4 = \text{Falso}$, se hace un intercambio:



- $5 < 9 = \text{Verdadero}$, no se realizan intercambios:



- $1 < 7 = \text{Verdadero}$, no se realizan cambios:



- $7 < 3 = \text{Falso}$, se hace un cambio:



- $7 < 6 = \text{Falso}$, se hace un cambio:



- $7 < 8 = \text{Verdadero}$, no se realizan intercambios:



5) Se repite el ordenamiento con comparaciones de la misma distancia:



- $2 < 3 = \text{Verdadero}$, no se realizan cambios:



- $3 < 4 = \text{Verdadero}$, no se realizan cambios:



- $4 < 5 = \text{Verdadero}$, no se realizan cambios:



- $5 < 9 = \text{Verdadero}$, no se realizan cambios:



- $1 < 3 = \text{Verdadero}$, no se realizan cambios:



- $3 < 6 = \text{Verdadero}$, no se realizan cambios:



- $6 < 7 = \text{Verdadero}$, no se realizan cambios:



- $7 < 8 = \text{Verdadero}$, no se realizan cambios:



6) Debido a que no hay cambios, se hace un ordenamiento de distancia igual a 1:



- $2 < 1 = \text{Falso}$, se hace intercambio:



- $2 < 3 = \text{Verdadero}$, no se hace intercambio:



- $3 < 3$ = Son iguales, por lo que se queda así:



- $3 < 4$ = Verdadero, no se hace intercambio:



- $4 < 6$ = Verdadero, no se hace intercambio:



- $6 < 5$ = Falso, se hace intercambio:



- $6 < 7$ = Verdadero, no se hace intercambio:



- $7 < 9$ = Verdadero, no se hace intercambio:



- $9 < 8$ = Falso, se hace intercambio:



7) Se repite el ordenamiento con la misma distancia.

Comparaciones:

- $1 < 2$ = Sí
- $2 < 3$ = Sí
- $3 < 3$ = Iguales
- $3 < 4$ = Sí
- $4 < 5$ = Sí
- $5 < 6$ = Sí
- $6 < 7$ = Sí

- $7 < 8 = \text{Sí}$
- $8 < 9 = \text{Sí}$

Las comparaciones acaban ya que al momento de dividir la distancia entre 2 no da un número entero, por lo que el proceso finaliza, dando como resultado un arreglo **ordenado**:

